

МИНИСТЕРСТВО ОБРАЗОВАНИЯ РЕСПУБЛИКИ БЕЛАРУСЬ

УЧРЕЖДЕНИЕ ОБРАЗОВАНИЯ

«БРЕСТСКИЙ ГОСУДАРСТВЕННЫЙ ТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ» ФАКУЛЬТЕТ

ЭЛЕКТРОННО-ИНФОРМАЦИОННЫХ СИСТЕМ

Кафедра интеллектуальных информационных технологий

Отчет по лабораторной работе №7

Специальность ПО-13

Выполнил:

А.С. Мышкевич

студент группы ПО-13

Проверил:

А.А. Крощенко

доц. кафедры ИИТ,

05.03.2026

Брест 2026

Цель работы: освоить возможности языка программирования Python в разработке оконных приложений

Задание 1. Построение графических примитивов и надписей

Требования к выполнению:

- Реализовать соответствующие классы, указанные в задании;
- Организовать ввод параметров для создания объектов (использовать экранные компоненты);
- Осуществить визуализацию графических примитивов

Важное замечание: должна быть предусмотрена возможность приостановки выполнения визуализации, изменения параметров «на лету» и снятия скриншотов с сохранением в текущую активную директорию.

б) Задать движение окружности по форме так, чтобы при касании границы окружность отражалась от нее.

Для всех динамических сцен необходимо задавать параметр скорости!

Выполнение:

Код программы

Circle.py

```
"""Класс движущейся окружности (для tkinter)."""

class MovingCircle: # pylint: disable=R0902
    """Класс окружности с движением и отражением от границ."""

    def __init__(self, canvas, x: int, y: int, radius: int, # pylint: disable=R0913,R0917
                 speed_x: float, speed_y: float, color: str):
        """
        Инициализация окружности.

        Args:
            canvas: Объект Canvas tkinter
            x: Начальная координата X
            y: Начальная координата Y
            radius: Радиус окружности
            speed_x: Скорость по X (пикселей в секунду)
            speed_y: Скорость по Y (пикселей в секунду)
            color: Цвет в формате строки (например, "red", "#FF0000")
        """
        self.canvas = canvas
        self.x = x
        self.y = y
        self.radius = radius
```

```
self.speed_x = speed_x
self.speed_y = speed_y
self.color = color
self.is_paused = False
self.oval_id = None
self._create_oval()
```

```
def _create_oval(self):
    """Создание окружности на canvas."""
    x1 = self.x - self.radius
    y1 = self.y - self.radius
    x2 = self.x + self.radius
    y2 = self.y + self.radius
    self.oval_id = self.canvas.create_oval(
        x1, y1, x2, y2,
        fill=self.color,
        outline='white',
        width=2
    )
```

```
def update(self, delta_time: float, width: int, height: int):
    """
```

Обновление позиции окружности с учетом отражения.

Args:

delta_time: Время с предыдущего кадра (секунды)

width: Ширина экрана

height: Высота экрана

"""

```
if self.is_paused:
    return
```

```
self.x += self.speed_x * delta_time
self.y += self.speed_y * delta_time
```

```
self._check_boundaries(width, height)
self._update_canvas_position()
```

```
def _check_boundaries(self, width: int, height: int):
```

"""Проверка столкновения с границами."""

Левая и правая граница

```
if self.x - self.radius <= 0:
```

```
    self.x = self.radius
```

```
    self.speed_x = abs(self.speed_x)
```

```
elif self.x + self.radius >= width:
```

```
    self.x = width - self.radius
```

```
    self.speed_x = -abs(self.speed_x)
```

```

# Верхняя и нижняя граница
if self.y - self.radius <= 0:
    self.y = self.radius
    self.speed_y = abs(self.speed_y)
elif self.y + self.radius >= height:
    self.y = height - self.radius
    self.speed_y = -abs(self.speed_y)

def _update_canvas_position(self):
    """Обновление позиции на canvas."""
    x1 = self.x - self.radius
    y1 = self.y - self.radius
    x2 = self.x + self.radius
    y2 = self.y + self.radius
    self.canvas.coords(self.oval_id, x1, y1, x2, y2)

def draw(self):
    """Отрисовка окружности на экране (обновление позиции)."""
    self._update_canvas_position()

def set_speed(self, speed_x: float, speed_y: float):
    """Установка скорости."""
    self.speed_x = speed_x
    self.speed_y = speed_y

def set_position(self, x: int, y: int):
    """Установка позиции."""
    self.x = x
    self.y = y

def set_radius(self, radius: int):
    """Установка радиуса."""
    self.radius = radius
    self.draw()

def set_color(self, color: str):
    """Установка цвета."""
    self.color = color
    self.canvas.itemconfig(self.oval_id, fill=color)

def toggle_pause(self):
    """Приостановка/возобновление движения."""
    self.is_paused = not self.is_paused
    return self.is_paused

def take_screenshot(self, filename: str = "screenshot") -> str:
    """
    Создание скриншота.

```

Args:

filename: Имя файла для сохранения

Returns:

Имя сохраненного файла

"""

```
if not hasattr(self, '_screenshot_count'):
    self._screenshot_count = 0 # pylint: disable=W0201
self._screenshot_count += 1
filename = f"{filename}_{self._screenshot_count:04d}.ps"
self.canvas.postscript(file=filename, colormode='color')
return filename
```

Main.py

"""Движущаяся окружность с отражением от границ (tkinter версия)."""

```
import tkinter as tk
from datetime import datetime
import os
```

```
class MovingCircle: # pylint: disable=R0902
```

"""Класс окружности с движением и отражением от границ."""

```
def __init__(self, canvas, x: int, y: int, radius: int, # pylint: disable=R0913,R0917
             speed_x: float, speed_y: float, color: str):
```

"""Инициализация окружности."""

```
self.canvas = canvas
self.x = x
self.y = y
self.radius = radius
self.speed_x = speed_x
self.speed_y = speed_y
self.color = color
self.is_paused = False
self._screenshot_count = 0
self.oval_id = None
self._create_oval()
```

```
def _create_oval(self):
```

"""Создание окружности на canvas."""

```
x1 = self.x - self.radius
y1 = self.y - self.radius
x2 = self.x + self.radius
y2 = self.y + self.radius
self.oval_id = self.canvas.create_oval(
    x1, y1, x2, y2, fill=self.color, outline='white', width=2
```

)

```
def update(self, delta_time: float, width: int, height: int):
    """Обновление позиции окружности."""
    if self.is_paused:
        return

    self.x += self.speed_x * delta_time
    self.y += self.speed_y * delta_time

    # Левая и правая граница
    if self.x - self.radius <= 0:
        self.x = self.radius
        self.speed_x = abs(self.speed_x)
    elif self.x + self.radius >= width:
        self.x = width - self.radius
        self.speed_x = -abs(self.speed_x)

    # Верхняя и нижняя граница
    if self.y - self.radius <= 0:
        self.y = self.radius
        self.speed_y = abs(self.speed_y)
    elif self.y + self.radius >= height:
        self.y = height - self.radius
        self.speed_y = -abs(self.speed_y)

    x1 = self.x - self.radius
    y1 = self.y - self.radius
    x2 = self.x + self.radius
    y2 = self.y + self.radius
    self.canvas.coords(self.oval_id, x1, y1, x2, y2)

def set_speed(self, speed_x: float, speed_y: float):
    """Установка скорости."""
    self.speed_x = speed_x
    self.speed_y = speed_y

def set_radius(self, radius: int):
    """Установка радиуса."""
    self.radius = radius
    x1 = self.x - self.radius
    y1 = self.y - self.radius
    x2 = self.x + self.radius
    y2 = self.y + self.radius
    self.canvas.coords(self.oval_id, x1, y1, x2, y2)

def set_color(self, color: str):
    """Установка цвета."""
```

```

        self.color = color
        self.canvas.itemconfig(self.oval_id, fill=color)

def toggle_pause(self):
    """Приостановка/возобновление движения."""
    self.is_paused = not self.is_paused
    return self.is_paused

def draw(self):
    """Отрисовка окружности."""
    x1 = self.x - self.radius
    y1 = self.y - self.radius
    x2 = self.x + self.radius
    y2 = self.y + self.radius
    self.canvas.coords(self.oval_id, x1, y1, x2, y2)

class CircleAnimationApp: # pylint: disable=R0902
    """Главное приложение."""

    def __init__(self, app_root): # pylint: disable=R0914
        """Инициализация приложения."""
        self.root = app_root
        self.root.title("Движущаяся окружность - Лабораторная работа №7")
        self.root.geometry("1000x700")
        self.root.configure(bg='#1e1e2e')

        # Параметры
        self.width = 700
        self.height = 600
        self.radius = 30
        self.speed_x = 5
        self.speed_y = 3
        self.color = "red"

        self._create_menu()
        self._create_canvas()
        self._create_controls()

        self.circle = MovingCircle(
            self.canvas, self.width // 2, self.height // 2,
            self.radius, self.speed_x, self.speed_y, self.color
        )

        self.is_running = True
        self.last_time = None
        self.animate()

```

```

def _create_menu(self):
    """Создание меню."""
    menubar = tk.Menu(self.root)
    self.root.config(menu=menubar)

    file_menu = tk.Menu(menubar, tearoff=0)
    menubar.add_cascade(label="Файл", menu=file_menu)
    file_menu.add_command(label="Скриншот", command=self.take_screenshot,
accelerator="Ctrl+S")
    file_menu.add_separator()
    file_menu.add_command(label="Выход", command=self.root.quit, accelerator="Ctrl+Q")

    self.root.bind('<Control-s>', lambda e: self.take_screenshot())
    self.root.bind('<Control-q>', lambda e: self.root.quit())

def _create_canvas(self):
    """Создание области для рисования."""
    canvas_frame = tk.Frame(self.root, bg='#1e1e2e')
    canvas_frame.pack(side=tk.LEFT, fill=tk.BOTH, expand=True, padx=10, pady=10)

    self.canvas = tk.Canvas(
        canvas_frame, width=self.width, height=self.height,
        bg='#1e1e2e', highlightthickness=2, highlightbackground='white'
    )
    self.canvas.pack(fill=tk.BOTH, expand=True)
    self.canvas.create_rectangle(0, 0, self.width, self.height, outline='white', width=2)

def _create_controls(self): # pylint: disable=R0914
    """Создание панели управления."""
    control_frame = tk.Frame(self.root, width=250, bg='#2e2e3e')
    control_frame.pack(side=tk.RIGHT, fill=tk.Y, padx=5, pady=10)
    control_frame.pack_propagate(False)

    tk.Label(control_frame, text="Управление", font=('Arial', 16, 'bold'),
        bg='#2e2e3e', fg='white').pack(pady=10)

    # Кнопки
    btn_frame = tk.Frame(control_frame, bg='#2e2e3e')
    btn_frame.pack(pady=10)

    self.pause_btn = tk.Button(btn_frame, text="⏸ Пауза", command=self.toggle_pause,
        width=10, bg='#444', fg='white')
    self.pause_btn.pack(side=tk.LEFT, padx=5)

    reset_btn = tk.Button(btn_frame, text="🔄 Сброс", command=self.reset,
        width=10, bg='#444', fg='white')
    reset_btn.pack(side=tk.LEFT, padx=5)

```



```

screenshot_btn = tk.Button(btn_frame, text="📷 Скриншот",
command=self.take_screenshot,
width=10, bg='#444', fg='white')
screenshot_btn.pack(side=tk.LEFT, padx=5)

tk.Frame(control_frame, height=2, bg='#555').pack(fill=tk.X, pady=10)

# Радиус
radius_frame = tk.Frame(control_frame, bg='#2e2e3e')
radius_frame.pack(fill=tk.X, pady=5, padx=10)
tk.Label(radius_frame, text="Радиус:", bg='#2e2e3e', fg='white').pack(anchor='w')
self.radius_var = tk.IntVar(value=self.radius)
radius_scale = tk.Scale(radius_frame, from_=10, to=100, orient=tk.HORIZONTAL,
variable=self.radius_var, command=self.change_radius,
bg='#2e2e3e', fg='white', highlightthickness=0)
radius_scale.pack(fill=tk.X)

# Скорость X
speed_x_frame = tk.Frame(control_frame, bg='#2e2e3e')
speed_x_frame.pack(fill=tk.X, pady=5, padx=10)
tk.Label(speed_x_frame, text="Скорость X:", bg='#2e2e3e',
fg='white').pack(anchor='w')
self.speed_x_var = tk.IntVar(value=self.speed_x)
speed_x_scale = tk.Scale(speed_x_frame, from_=1, to=50, orient=tk.HORIZONTAL,
variable=self.speed_x_var, command=self.change_speed_x,
bg='#2e2e3e', fg='white', highlightthickness=0)
speed_x_scale.pack(fill=tk.X)

# Скорость Y
speed_y_frame = tk.Frame(control_frame, bg='#2e2e3e')
speed_y_frame.pack(fill=tk.X, pady=5, padx=10)
tk.Label(speed_y_frame, text="Скорость Y:", bg='#2e2e3e',
fg='white').pack(anchor='w')
self.speed_y_var = tk.IntVar(value=self.speed_y)
speed_y_scale = tk.Scale(speed_y_frame, from_=1, to=50, orient=tk.HORIZONTAL,
variable=self.speed_y_var, command=self.change_speed_y,
bg='#2e2e3e', fg='white', highlightthickness=0)
speed_y_scale.pack(fill=tk.X)

tk.Frame(control_frame, height=2, bg='#555').pack(fill=tk.X, pady=10)

# Цвета
tk.Label(control_frame, text="Цвет:", bg='#2e2e3e', fg='white').pack(anchor='w',
padx=10)
color_frame = tk.Frame(control_frame, bg='#2e2e3e')
color_frame.pack(pady=5, padx=10)

colors = [("● Красный", "red"), ("■ Зеленый", "green"), ("● Синий", "blue"),

```

```

        ("Желтый", "yellow"), ("Фиолетовый", "purple"),
        ("Оранжевый", "orange")]

    for text, color in colors:
        btn = tk.Button(color_frame, text=text, command=lambda c=color:
self.change_color(c),
                        bg='#444', fg='white', width=12)
        btn.pack(pady=2)

    tk.Frame(control_frame, height=2, bg='#555').pack(fill=tk.X, pady=10)

    # Информация
    self.info_label = tk.Label(control_frame, text="", bg='#2e2e3e', fg='white',
                                justify=tk.LEFT, font=('Courier', 10))
    self.info_label.pack(pady=10)

    # Подсказки
    tip_text = ("Подсказки:\n• Пауза/Старт - Пробел\n"
                "• Ctrl+S - Скриншот\n• Ctrl+Q - Выход")
    tk.Label(control_frame, text=tip_text, bg='#2e2e3e', fg='#aaa',
              font=('Arial', 9), justify=tk.LEFT).pack(side=tk.BOTTOM, pady=20)

    self.root.bind('<space>', lambda e: self.toggle_pause())

def change_radius(self, value):
    """Изменение радиуса."""
    self.radius = int(value)
    self.circle.set_radius(self.radius)

def change_speed_x(self, value):
    """Изменение скорости X."""
    self.speed_x = int(value)
    self.circle.set_speed(self.speed_x, self.speed_y)

def change_speed_y(self, value):
    """Изменение скорости Y."""
    self.speed_y = int(value)
    self.circle.set_speed(self.speed_x, self.speed_y)

def change_color(self, color):
    """Изменение цвета."""
    self.color = color
    self.circle.set_color(color)

def toggle_pause(self):
    """Переключение паузы."""
    paused = self.circle.toggle_pause()
    self.pause_btn.config(text="▶ Старт" if paused else "⏸ Пауза")

```

```

def reset(self):
    """Сброс параметров."""
    self.radius = 30
    self.speed_x = 5
    self.speed_y = 3
    self.color = "red"

    self.radius_var.set(30)
    self.speed_x_var.set(5)
    self.speed_y_var.set(3)

    self.circle.set_radius(30)
    self.circle.set_speed(5, 3)
    self.circle.set_color("red")

    if self.circle.is_paused:
        self.circle.is_paused = False
        self.pause_btn.config(text="⏸ Пауза")

    self.circle.x = self.width // 2
    self.circle.y = self.height // 2
    self.circle.draw()

def take_screenshot(self):
    """Создание скриншота."""
    os.makedirs("screenshots", exist_ok=True)
    timestamp = datetime.now().strftime("%Y%m%d_%H%M%S")
    filename = f"screenshots/screenshot_{timestamp}.eps"
    self.canvas.postscript(file=filename, colormode='color')
    print(f"Скриншот сохранен: {filename}")

def update_info(self):
    """Обновление информационной панели."""
    status = "⏸ ПАУЗА" if self.circle.is_paused else "▶ ДВИЖЕНИЕ"
    text = (f"Статус: {status}\nПозиция: ({int(self.circle.x)}, {int(self.circle.y)})\n"
           f"Радиус: {self.circle.radius}\nСкорость: ({self.circle.speed_x}, {self.circle.speed_y})")
    self.info_label.config(text=text)

def animate(self):
    """Основной цикл анимации."""
    if not self.is_running:
        return

    current_time = self.root.tk.call('clock', 'milliseconds')
    if self.last_time is None:
        self.last_time = current_time

```

```

delta_time = (current_time - self.last_time) / 1000.0
self.last_time = current_time

delta_time = min(delta_time, 0.05)

self.circle.update(delta_time, self.width, self.height)
self.update_info()
self.root.after(20, self.animate)

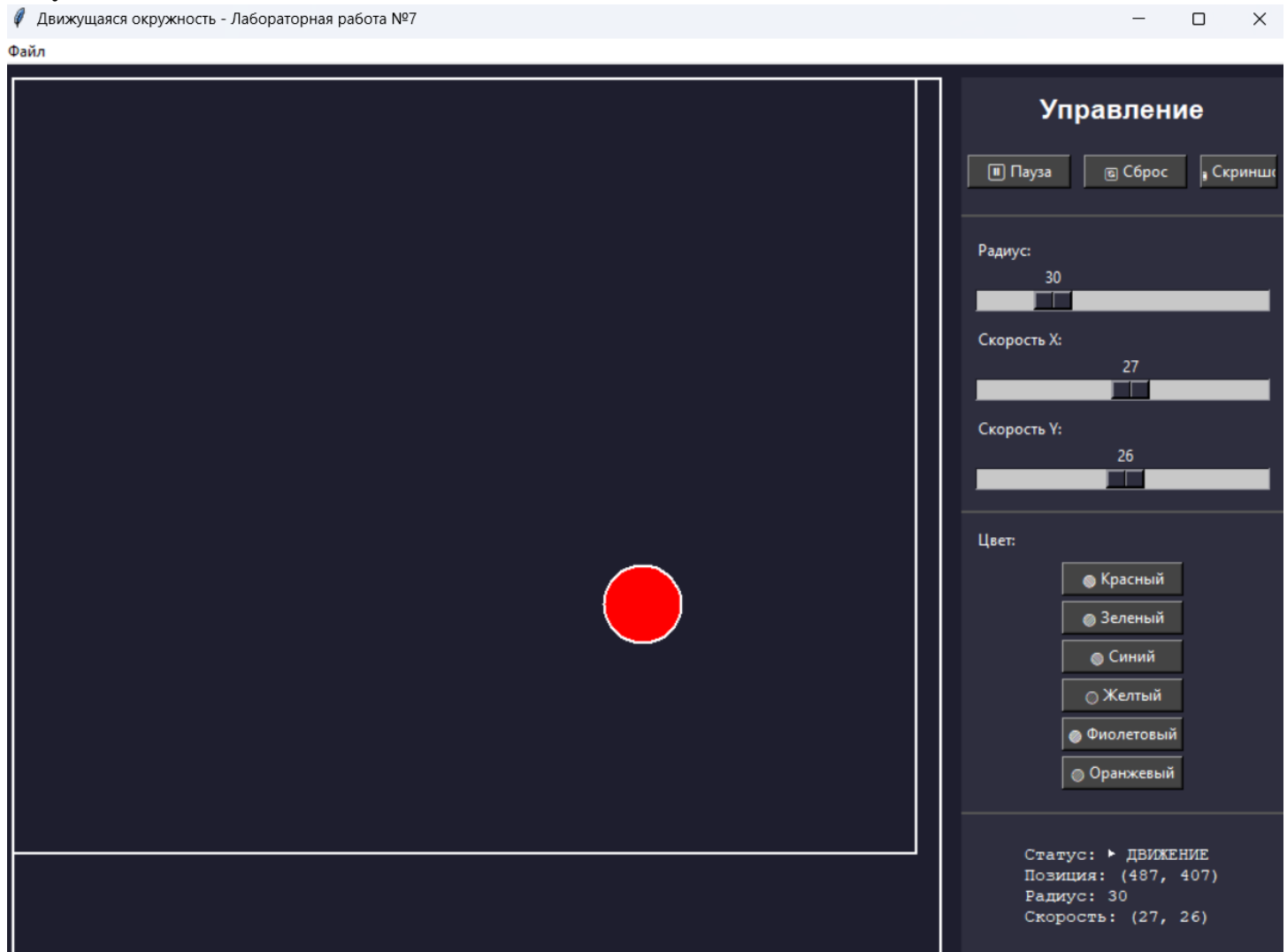
```

```

if __name__ == "__main__":
    ROOT = tk.Tk()
    CircleAnimationApp(ROOT)
    ROOT.mainloop()

```

Результат выполнения:




```
# Углы для левой и правой ветки с учетом ветра
left_angle = angle - self.left_angle + self.wind * depth * 0.5
right_angle = angle + self.right_angle + self.wind * depth * 0.5
```

```
new_length = length * self.length_ratio
self.draw(x2, y2, new_length, left_angle, depth + 1)
self.draw(x2, y2, new_length, right_angle, depth + 1)
```

```
def _get_color(self, depth: int) -> str:
    """Получение цвета в зависимости от глубины и режима."""
    if self.color_mode == "gradient":
        if depth < 4:
            return "#5D3A1A"
        if depth < 7:
            return "#8B5A2B"
        if depth < 10:
            return "#228B22"
        return "#32CD32"
    if self.color_mode == "rainbow":
        colors = ["#FF4444", "#FF8844", "#FFFF44", "#44FF44",
                  "#44FFFF", "#4444FF", "#FF44FF"]
        return colors[depth % len(colors)]
    return "#8B4513" if depth < 4 else "#228B22"
```

```
def redraw(self):
    """Перерисовка дерева."""
    self.canvas.delete("all")
    self.draw(self.x, self.y, self.length, self.angle, 0)
```

```
class TreeApp: # pylint: disable=R0902
    """Главное приложение."""
```

```
def __init__(self, app_root): # pylint: disable=R0914
    """Инициализация приложения."""
    self.root = app_root
    self.root.title("Склоненное дерево Пифагора - Обдуваемое ветром")
    self.root.geometry("1100x750")
    self.root.configure(bg='#1e1e2e')

    # Параметры по умолчанию
    self.canvas_width = 900
    self.canvas_height = 700
    self.start_x = self.canvas_width // 2
    self.start_y = self.canvas_height - 50
    self.length = 100
    self.angle = -90
```

```

self.wind = 0
self.max_depth = 11
self.left_angle = 45
self.right_angle = 45
self.length_ratio = 0.68
self.color_mode = "gradient"

# Атрибуты для элементов управления (инициализация в __init__)
self.length_var = None
self.depth_var = None
self.left_var = None
self.right_var = None
self.ratio_var = None
self.wind_var = None
self.color_mode_var = None
self.animate_btn = None
self.info_label = None

self._create_menu()
self._create_canvas()
self._create_controls()

self.tree = PythagoreanTree(
    self.canvas, self.start_x, self.start_y,
    self.length, self.angle, self.wind
)
self.tree.max_depth = self.max_depth
self.tree.left_angle = self.left_angle
self.tree.right_angle = self.right_angle
self.tree.length_ratio = self.length_ratio
self.tree.color_mode = self.color_mode
self.tree.redraw()

self.wind_animation = False

def _create_menu(self):
    """Создание меню."""
    menubar = tk.Menu(self.root)
    self.root.config(menu=menubar)

    file_menu = tk.Menu(menubar, tearoff=0)
    menubar.add_cascade(label="Файл", menu=file_menu)
    file_menu.add_command(label="Скриншот", command=self.take_screenshot,
                          accelerator="Ctrl+S")
    file_menu.add_separator()
    file_menu.add_command(label="Выход", command=self.root.quit,
                          accelerator="Ctrl+Q")

```

```

self.root.bind('<Control-s>', lambda e: self.take_screenshot())
self.root.bind('<Control-q>', lambda e: self.root.quit())

def _create_canvas(self):
    """Создание области для рисования."""
    canvas_frame = tk.Frame(self.root, bg='#1e1e2e')
    canvas_frame.pack(side=tk.LEFT, fill=tk.BOTH,
                      expand=True, padx=10, pady=10)

    self.canvas = tk.Canvas(
        canvas_frame, width=self.canvas_width, height=self.canvas_height,
        bg='#1a3a1a', highlightthickness=2, highlightbackground='white'
    )
    self.canvas.pack(fill=tk.BOTH, expand=True)

    # Рисуем землю
    self.canvas.create_rectangle(0, self.canvas_height - 25,
                                self.canvas_width, self.canvas_height,
                                fill="#5D3A1A", outline="")

def _create_controls(self): # pylint: disable=R0914,R0915
    """Создание панели управления."""
    control_frame = tk.Frame(self.root, width=270, bg='#2e2e3e')
    control_frame.pack(side=tk.RIGHT, fill=tk.Y, padx=5, pady=10)
    control_frame.pack_propagate(False)

    tk.Label(control_frame, text="🌲 Параметры дерева",
             font=('Arial', 15, 'bold'), bg='#2e2e3e',
             fg='white').pack(pady=10)

    self._add_length_control(control_frame)
    self._add_depth_control(control_frame)
    self._add_left_angle_control(control_frame)
    self._add_right_angle_control(control_frame)
    self._add_ratio_control(control_frame)
    self._add_wind_control(control_frame)
    self._add_color_control(control_frame)
    self._add_buttons(control_frame)
    self._add_info_label(control_frame)
    self._add_tips(control_frame)

    self.root.bind('<space>', lambda e: self.toggle_animation())

def _add_length_control(self, parent):
    """Добавление контроля длины ствола."""
    frame = tk.Frame(parent, bg='#2e2e3e')
    frame.pack(fill=tk.X, pady=5, padx=10)
    tk.Label(frame, text="📏 Длина ствола:", bg='#2e2e3e',

```



```

        fg='white').pack(anchor='w')
self.length_var = tk.IntVar(value=self.length)
scale = tk.Scale(frame, from_=60, to=160, orient=tk.HORIZONTAL,
                 variable=self.length_var, command=self.change_length,
                 bg='#2e2e3e', fg='white', highlightthickness=0)
scale.pack(fill=tk.X)

def _add_depth_control(self, parent):
    """Добавление контроля глубины рекурсии."""
    frame = tk.Frame(parent, bg='#2e2e3e')
    frame.pack(fill=tk.X, pady=5, padx=10)
    tk.Label(frame, text="Глубина рекурсии:", bg='#2e2e3e',
             fg='white').pack(anchor='w')
    self.depth_var = tk.IntVar(value=self.max_depth)
    scale = tk.Scale(frame, from_=5, to=15, orient=tk.HORIZONTAL,
                     variable=self.depth_var, command=self.change_depth,
                     bg='#2e2e3e', fg='white', highlightthickness=0)
    scale.pack(fill=tk.X)

def _add_left_angle_control(self, parent):
    """Добавление контроля левого угла."""
    frame = tk.Frame(parent, bg='#2e2e3e')
    frame.pack(fill=tk.X, pady=5, padx=10)
    tk.Label(frame, text="← Угол левой ветки:", bg='#2e2e3e',
             fg='white').pack(anchor='w')
    self.left_var = tk.IntVar(value=self.left_angle)
    scale = tk.Scale(frame, from_=20, to=70, orient=tk.HORIZONTAL,
                     variable=self.left_var, command=self.change_left_angle,
                     bg='#2e2e3e', fg='white', highlightthickness=0)
    scale.pack(fill=tk.X)

def _add_right_angle_control(self, parent):
    """Добавление контроля правого угла."""
    frame = tk.Frame(parent, bg='#2e2e3e')
    frame.pack(fill=tk.X, pady=5, padx=10)
    tk.Label(frame, text="→ Угол правой ветки:", bg='#2e2e3e',
             fg='white').pack(anchor='w')
    self.right_var = tk.IntVar(value=self.right_angle)
    scale = tk.Scale(frame, from_=20, to=70, orient=tk.HORIZONTAL,
                     variable=self.right_var, command=self.change_right_angle,
                     bg='#2e2e3e', fg='white', highlightthickness=0)
    scale.pack(fill=tk.X)

def _add_ratio_control(self, parent):
    """Добавление контроля коэффициента уменьшения."""
    frame = tk.Frame(parent, bg='#2e2e3e')
    frame.pack(fill=tk.X, pady=5, padx=10)
    tk.Label(frame, text="Кэф. уменьшения:", bg='#2e2e3e',

```

```

        fg='white').pack(anchor='w')
self.ratio_var = tk.DoubleVar(value=self.length_ratio)
scale = tk.Scale(frame, from_=0.55, to=0.8, resolution=0.01,
                 orient=tk.HORIZONTAL, variable=self.ratio_var,
                 command=self.change_ratio, bg='#2e2e3e',
                 fg='white', highlightthickness=0)
scale.pack(fill=tk.X)

def _add_wind_control(self, parent):
    """Добавление контроля силы ветра."""
    frame = tk.Frame(parent, bg='#2e2e3e')
    frame.pack(fill=tk.X, pady=5, padx=10)
    tk.Label(frame, text="🌀 Сила ветра:", bg='#2e2e3e',
             fg='white').pack(anchor='w')
    self.wind_var = tk.IntVar(value=self.wind)
    scale = tk.Scale(frame, from_=-25, to=25, orient=tk.HORIZONTAL,
                     variable=self.wind_var, command=self.change_wind,
                     bg='#2e2e3e', fg='white', highlightthickness=0)
    scale.pack(fill=tk.X)

def _add_color_control(self, parent):
    """Добавление выбора режима цвета."""
    frame = tk.Frame(parent, bg='#2e2e3e')
    frame.pack(fill=tk.X, pady=5, padx=10)
    tk.Label(frame, text="🌈 Режим цвета:", bg='#2e2e3e',
             fg='white').pack(anchor='w')
    self.color_mode_var = tk.StringVar(value=self.color_mode)
    color_menu = ttk.Combobox(frame, textvariable=self.color_mode_var,
                              values=["gradient", "rainbow"],
                              state="readonly")
    color_menu.pack(fill=tk.X)
    color_menu.bind('<<ComboboxSelected>>', self.change_color_mode)

def _add_buttons(self, parent):
    """Добавление кнопок управления."""
    btn_frame = tk.Frame(parent, bg='#2e2e3e')
    btn_frame.pack(pady=10)

    self.animate_btn = tk.Button(btn_frame, text="🎬 Анимация ветра",
                                 command=self.toggle_animation,
                                 bg='#444', fg='white', width=18)
    self.animate_btn.pack(pady=3)

    reset_btn = tk.Button(btn_frame, text="🔄 Сброс параметров",
                           command=self.reset_params,
                           bg='#444', fg='white', width=18)
    reset_btn.pack(pady=3)

```

```

screenshot_btn = tk.Button(btn_frame, text="📷 Скриншот",
                             command=self.take_screenshot,
                             bg='#444', fg='white', width=18)
screenshot_btn.pack(pady=3)

def _add_info_label(self, parent):
    """Добавление информационной метки."""
    self.info_label = tk.Label(parent, text="", bg='#2e2e3e',
                                fg='white', justify=tk.LEFT,
                                font=('Courier', 9))
    self.info_label.pack(pady=10)

def _add_tips(self, parent):
    """Добавление подсказок."""
    tip_text = ("💡 Подсказки:\n• Ctrl+S - Скриншот\n"
                "• Ctrl+Q - Выход")
    tk.Label(parent, text=tip_text, bg='#2e2e3e', fg='#aaa',
              font=('Arial', 9), justify=tk.LEFT).pack(side=tk.BOTTOM,
                                                         pady=20)

def change_length(self, value):
    """Изменение длины ствола."""
    self.length = int(value)
    self.tree.length = self.length
    self.tree.redraw()
    self.update_info()

def change_depth(self, value):
    """Изменение глубины рекурсии."""
    self.max_depth = int(value)
    self.tree.max_depth = self.max_depth
    self.tree.redraw()
    self.update_info()

def change_left_angle(self, value):
    """Изменение угла левой ветки."""
    self.left_angle = int(value)
    self.tree.left_angle = self.left_angle
    self.tree.redraw()
    self.update_info()

def change_right_angle(self, value):
    """Изменение угла правой ветки."""
    self.right_angle = int(value)
    self.tree.right_angle = self.right_angle
    self.tree.redraw()
    self.update_info()

```

```

def change_ratio(self, value):
    """Изменение коэффициента уменьшения."""
    self.length_ratio = float(value)
    self.tree.length_ratio = self.length_ratio
    self.tree.redraw()
    self.update_info()

def change_wind(self, value):
    """Изменение силы ветра."""
    self.wind = int(value)
    self.tree.wind = self.wind
    self.tree.redraw()
    self.update_info()

def change_color_mode(self, event=None): # pylint: disable=W0613
    """Изменение режима цвета."""
    self.color_mode = self.color_mode_var.get()
    self.tree.color_mode = self.color_mode
    self.tree.redraw()

def toggle_animation(self):
    """Включение/выключение анимации ветра."""
    self.wind_animation = not self.wind_animation
    text = "⏏ Остановить ветер" if self.wind_animation else "🌀 Анимация ветра"
    self.animate_btn.config(text=text)
    if self.wind_animation:
        self.animate_wind()

def animate_wind(self):
    """Анимация ветра."""
    if not self.wind_animation:
        return
    t = time.time() * 1.5
    wind_value = int(math.sin(t) * 20)
    self.wind_var.set(wind_value)
    self.change_wind(wind_value)
    self.root.after(80, self.animate_wind)

def reset_params(self):
    """Сброс параметров."""
    self.length = 100
    self.max_depth = 11
    self.left_angle = 45
    self.right_angle = 45
    self.length_ratio = 0.68
    self.wind = 0
    self.color_mode = "gradient"

```

```

self.length_var.set(100)
self.depth_var.set(11)
self.left_var.set(45)
self.right_var.set(45)
self.ratio_var.set(0.68)
self.wind_var.set(0)
self.color_mode_var.set("gradient")

```

```

self.tree.length = 100
self.tree.max_depth = 11
self.tree.left_angle = 45
self.tree.right_angle = 45
self.tree.length_ratio = 0.68
self.tree.wind = 0
self.tree.color_mode = "gradient"
self.tree.redraw()
self.update_info()

```

```

def take_screenshot(self):
    """Создание скриншота."""
    os.makedirs("screenshots", exist_ok=True)
    timestamp = datetime.now().strftime("%Y%m%d_%H%M%S")
    filename = f"screenshots/tree_{timestamp}.eps"
    self.canvas.postscript(file=filename, colormode='color')
    print(f"Скриншот сохранен: {filename}")

```

```

def update_info(self):
    """Обновление информационной панели."""
    text = (f"📏 Длина: {self.length}\n📏 Глубина: {self.max_depth}\n"
           f"↩️ Левый угол: {self.left_angle}°\n➡️ Правый угол: {self.right_angle}°\n"
           f"📐 Коэф. уменьшения: {self.length_ratio:.2f}\n🌬️ Ветер: {self.wind}")
    self.info_label.config(text=text)

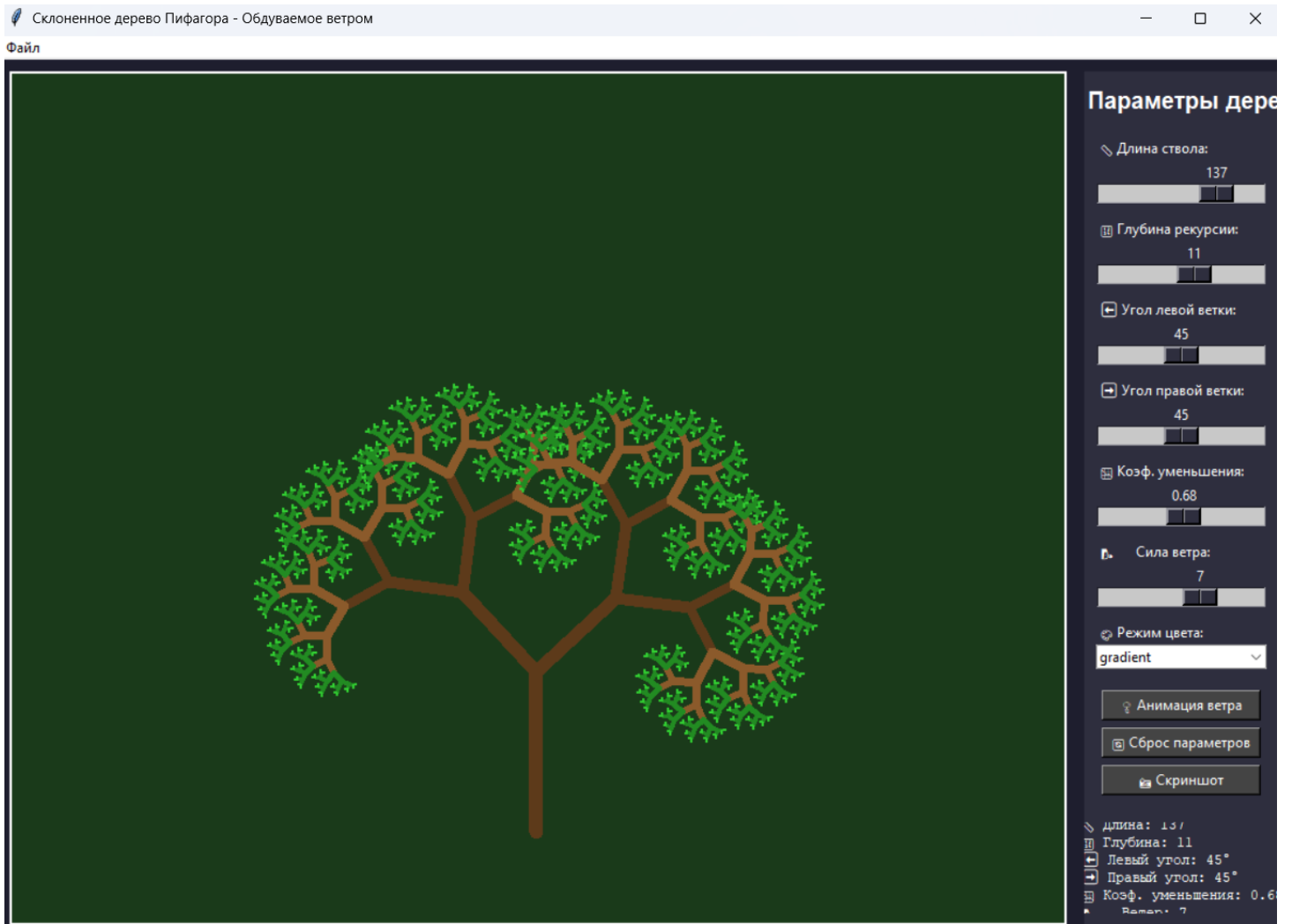
```

```

if __name__ == "__main__":
    ROOT = tk.Tk()
    TreeApp(ROOT)
    ROOT.mainloop()

```

Результат выполнения:



Вывод: освоил возможности языка программирования Python в разработке оконных приложений